

Guardian Battery ECM Project

Full System Documentation

Cell digital twin • 2RC Thevenin ECM • Coulomb Counting • EKF SOC • recursive TLS R0/SOH • $T_s = 0.1 \text{ s}$

Reference cell: EVE LF50K, 3.2 V / 50 Ah LiFePO4

Prepared as a reference document for the Guardian_Battery model reference (Guardian robot integration)

Author: Ahmed Hassan

September 2026

Table of Contents

Table of Contents.....	2
1. Purpose and Scope.....	4
1.1 What changed from the original course specification.....	4
2. System Architecture.....	4
2.1 Top-level model — Guardian_Battery	4
2.2 DigitalTwin_and_SOC_SOH.....	5
2.3 Cell_DigitalTwin (plant model).....	5
2.3.1 CoulombCounting	5
2.3.2 RCs_branches (2RC Thevenin)	6
2.3.3 OCV_Hyst	6
2.4 SOCandSOH (estimation stage).....	6
2.4.1 TLS_RO_SOH as a Variant Subsystem	7
3. Sample Time (Ts).....	7
3.1 Why Ts was changed.....	7
3.2 What had to be rescaled	7
4. Algorithms.....	8
4.1 2RC discrete Thevenin model	8
4.2 Coulomb Counting SOC.....	8
4.3 OCV + hysteresis noise	9
4.4 EKF SOC estimator (2RC, 3-state, live R0)	9
4.5 TLS R0/SOH estimator — Coupled variant.....	10
4.6 TLS R0/SOH estimator — Open-loop variant.....	12
4.7 Variant Subsystem configuration.....	13
5. Bus Objects.....	13
5.1 GuardianCellBus — inner Cell_data bus (8 elements)	14
5.2 BatteryDataBus — public Battery_data boundary (5 elements)	14
6. Base Workspace Script.....	14
7. Validation Results.....	16
7.1 SOC: Coulomb Counting vs. EKF.....	16
7.2 R0 identification	17
7.3 SOH ratio	17
7.4 Coupled vs. Open-loop — observed result and analysis.....	18
8. Development History and Key Fixes	18
8.1 Bus object mismatch after variable renaming	18
8.2 Bus hierarchy mismatch.....	19
8.3 Recursive TLS forgetting factor not rescaled for Ts	19

8.4 R0_cell is a comparison signal only, not part of the estimation loop.....	19
8.5 Fast Restart and dialog changes not taking effect	19

(Right-click the table above and choose "Update Field" in Word to populate page numbers.)

1. Purpose and Scope

This document is the full reference documentation for the Guardian_Battery Simulink model: a discrete-time digital twin of a single LiFePO₄ cell (EVE LF50K, 3.2 V / 50 Ah), combined with a Coulomb Counting SOC estimator, an Extended Kalman Filter (EKF) SOC/polarization estimator, and a recursive Total Least Squares (TLS) online R₀/SOH identifier. It captures the model's architecture, algorithms, sample-time discretization, bus interfaces, and validated results, so the model and this project can be picked back up, shared, or extended — including as the reference this documentation is intended to support: integrating Guardian_Battery as a Model Reference into the larger Guardian robot project.

The model originated from the Guardian Battery ECM Project course specification (Simulink for Model-Based Design, Module 3), whose Core tier (1RC Thevenin ECM, OCV + hysteresis noise, Coulomb Counting SOC) and Bonus tiers (2RC extension, EKF SOC estimator, TLS-based R₀ identification + SOH ratio) form the technical basis of everything described here. This document covers the subsequent redesign of that project into an integrated, closed-loop estimation architecture running at $T_s = 0.1$ s.

1.1 What changed from the original course specification

- The three Bonus tiers (2RC extension, EKF, TLS/SOH) were integrated into a single closed-loop architecture instead of three independent, isolated additions.
- The EKF was extended from a 1RC/2-state filter to a 2RC/3-state filter, and R₀ was promoted from a fixed datasheet constant to a live, TLS-estimated input.
- The TLS R₀ identifier was converted from a single end-of-run batch estimate (one SVD over the whole logged drive cycle) to a recursive, continuously-updating online estimator.
- The model's sample time was changed from $T_s = 1$ s (matching the original 1 Hz current dataset) to $T_s = 0.1$ s, to better resolve the R2C2 branch's fast (~ 1.8 s) time constant.
- Two interchangeable implementations of the TLS/SOH block now exist — a fully-coupled version and an open-loop version — selectable via a Variant Subsystem, both documented in Section 4.

2. System Architecture

The model is organized top-down: a top-level model exposing the public interface, a plant + estimation subsystem, and within it the cell digital twin (plant) and the SOC/SOH estimators.

2.1 Top-level model — Guardian_Battery

The top level takes the drive-cycle current from workspace data and exposes a single composite output, Battery_data, carrying everything a caller of this model needs.

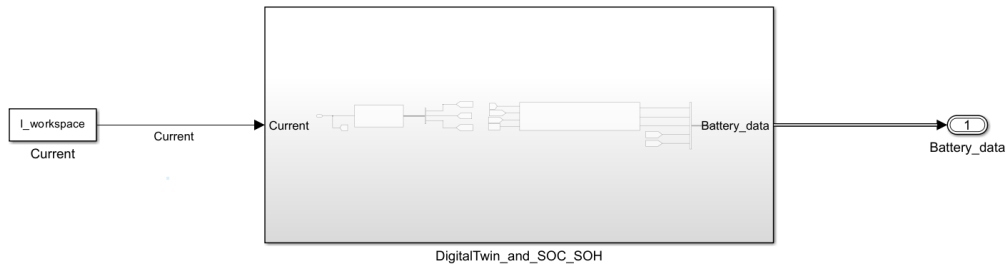


Figure 1. Top-level Guardian_Battery model. Current_A drives DigitalTwin_and_SOC_SOH; the subsystem's single output, Battery_data, is the model's public interface.

2.2 DigitalTwin_and_SOC_SOH

This subsystem contains the plant (Cell_DigitalTwin) and the estimation stage (SOCandSOH), and assembles their outputs into the public Battery_data bus.

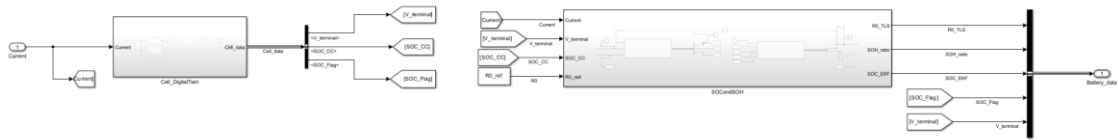


Figure 2. DigitalTwin_and_SOC_SOH: Cell_DigitalTwin (plant) feeds V_terminal, SOC_CC, and SOC_Flag into SOCandSOH, which produces R0_TLS, SOH_ratio, and SOC_EKF.

2.3 Cell_DigitalTwin (plant model)

The plant model implements the 2RC Thevenin equivalent circuit: Coulomb Counting for SOC, the two RC polarization branches, the R0 ohmic drop, and the OCV(SOC) lookup with hysteresis noise, summed into V_terminal.

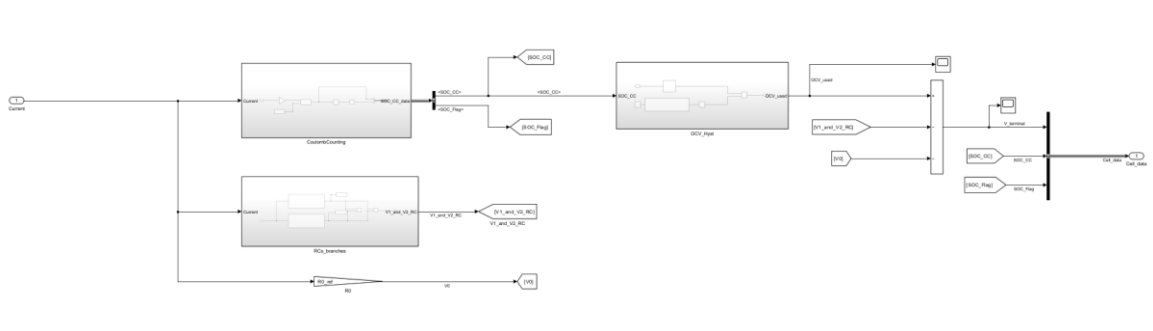


Figure 3. Cell_DigitalTwin internals: CoulombCounting and RCs_branches run in parallel from Current; OCV_Hyst consumes SOC_CC; all three combine into V_terminal, packaged into the Cell_data bus.

2.3.1 CoulombCounting

Integrates current into SOC using a discrete integrator with gain $-100/Q$, clamps to the [0,100]% (operationally [10,90]% per the datasheet's recommended window) and raises SOC_Flag on out-of-window events.

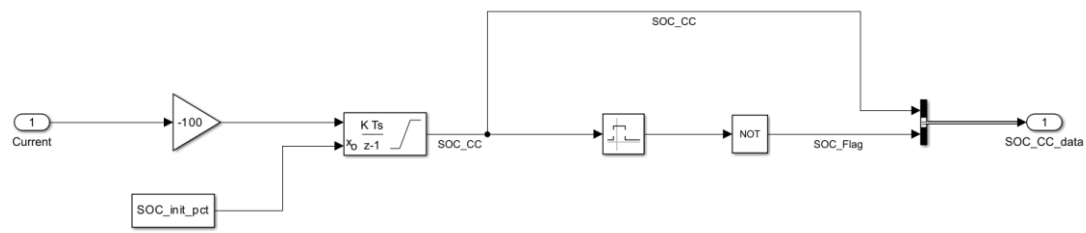


Figure 4. CoulombCounting: $SOC(k+1) = SOC(k) - (Ts/(Q \cdot 3600)) \cdot I(k) \cdot 100$, implemented as a gain into a discrete integrator, with a Relay + NOT producing SOC_Flag.

2.3.2 RCs_branches (2RC Thevenin)

Two independent RC branches (R1C1 and R2C2) run in parallel from the same current input and sum into the combined polarization voltage V1_and_V2_RC.

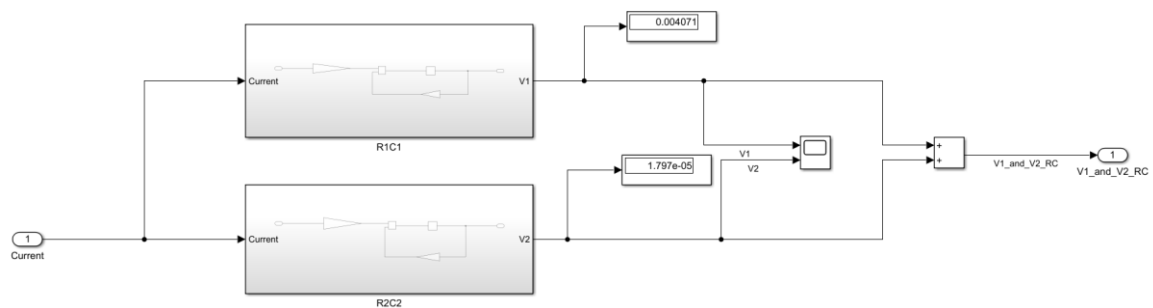


Figure 5. RCs_branches: R1C1 and R2C2 subsystems (each an exact ZOH discretization, $a1/a2 = \exp(-Ts/(R \cdot C))$) sum their V1 and V2 outputs into V1_and_V2_RC.

2.3.3 OCV_Hyst

Looks up OCV(SOC) via a 1-D lookup table and adds a bounded hysteresis noise term (Band-Limited White Noise $\rightarrow$ Discrete Filter $\rightarrow$ Saturation), per the original spec's FR-3, rescaled for $Ts = 0.1$ s (Section 3).

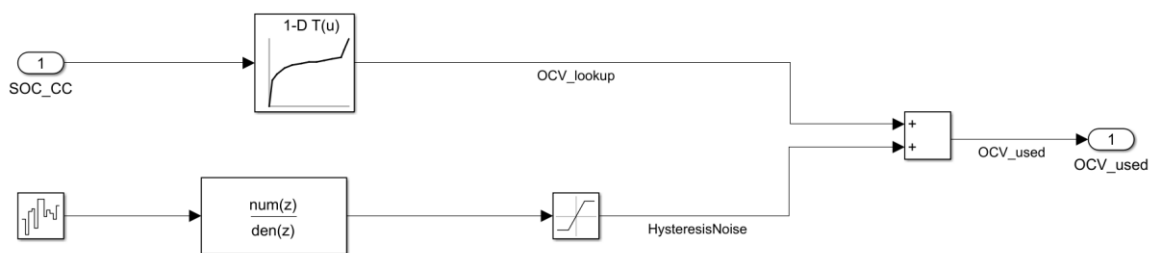


Figure 6. OCV_Hyst: 1-D lookup table for OCV(SOC_CC), summed with a filtered, saturated Band-Limited White Noise term to produce OCV_used.

2.4 SOCandSOH (estimation stage)

This subsystem holds the EKF_SOC and TLS_RO_SOH blocks, wired into a closed loop: the EKF's SOC_hat, V1_hat, and V2_hat feed TLS_RO_SOH, and TLS_RO_SOH's R0_TLS feeds back into the

EKF's $R0_in$ through a one-sample Unit Delay, which is what breaks the algebraic loop between the two blocks.

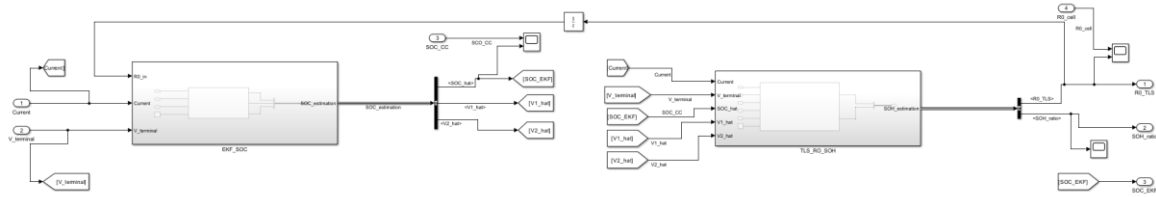


Figure 7. SOCandSOH: EKF_SOC (2RC, 3-state, live $R0$) and TLS_RO_SOH (recursive online TLS) closing the loop through a Unit Delay ($1/z$) on $R0$. $R0_cell$ (external, datasheet-referenced) is logged alongside $R0_TLS$ for comparison only — it is not part of the estimation loop.

2.4.1 TLS_RO_SOH as a Variant Subsystem

TLS_RO_SOH is implemented as a Variant Subsystem with two interchangeable choices — Coupled and Open_loop — selected by the Model Workspace variable `TLS_impl_select` (1 = Coupled, 2 = Open_loop), evaluated at Update Diagram time. Both variants share the same external port set (the union of what either implementation needs); the Open_loop variant simply does not route $V1_hat/V2_hat$ any further internally. See Section 4.7 for the full setup.

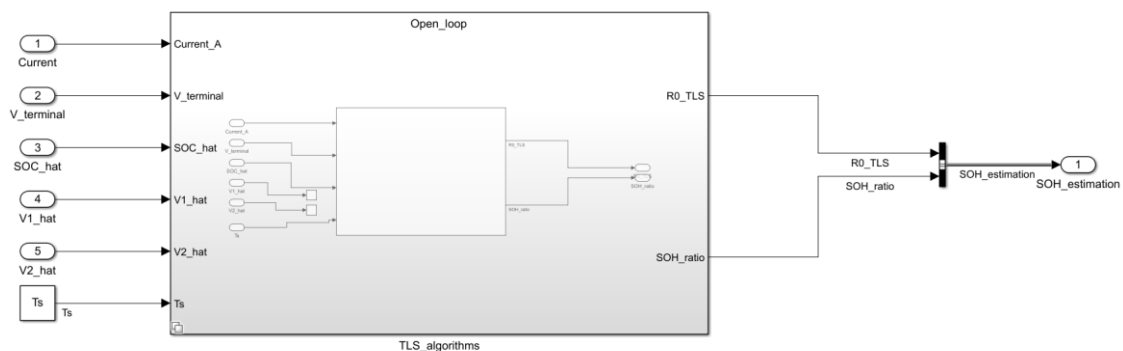


Figure 8. Inside the Open_loop variant choice: Current, $V_terminal$, SOC_hat , $V1_hat$, $V2_hat$, and Ts enter `TLS_algorithms`; only $R0_TLS$ and SOH_ratio are produced ($V1_hat/V2_hat$ are not used internally by this variant).

3. Sample Time (Ts)

The model originally ran at $Ts = 1$ s, matching the native 1 Hz rate of the provided drive-cycle current dataset (per the course spec's FR-1 and Sec. 9.3). It was changed to $Ts = 0.1$ s partway through this project.

3.1 Why Ts was changed

The R2C2 branch's time constant is $\tau_2 = R_2 \cdot C_2 \approx 1.8$ s. At $Ts = 1$ s that is only ~ 1.8 samples per time constant — borderline for resolving that branch's dynamics, even though the discrete update itself ($a_2 = \exp(-Ts/\tau_2)$) is an exact zero-order-hold discretization for any Ts . At $Ts = 0.1$ s, the same branch gets ~ 18 samples per time constant, meaningfully improving R2C2 fidelity.

3.2 What had to be rescaled

Several quantities in this project were tuned or specified at a particular sample rate and do not automatically stay correct if T_s changes without also rescaling them. The general pattern used throughout: for a quantity defined at $T_{s_ref} = 1$ s as a pole/decay coefficient a_ref , the T_s -consistent rescaled coefficient is $a(T_s) = a_ref^{(T_s/T_{s_ref})}$ — this preserves the same real-time decay/bandwidth at any T_s , because it is the same $\exp(-T_s/\tau)$ relationship evaluated consistently.

Quantity	Where	At $T_s=1$ s	Rescale formula	At $T_s=0.1$ s
a_1, a_2 (RC branch poles)	Plant, EKF, TLS	$\exp(-T_s/\tau)$ — always T_s -parametrized	$a = \exp(-T_s/(R \cdot C))$	auto-adapts
Hysteresis filter (FR-3)	OCV_Hyst $\rightarrow$ Discrete Filter	num=[0.02], den=[1,-0.98]	$a=0.98^{(T_s/1)}$; num=1-a; den=[1,-a]	num $\approx$ 0.00202, den $\approx$ [1,-0.99798]
Band-Limited White Noise sample time	OCV_Hyst	1	= T_s	0.1
EKF process noise Qcov	EKF_SOC_Estimator.m	diag([1e-3,1e-6,1e-6])	$\times (T_s/1)$	diag([1e-4,1e-7,1e-7])
TLS forgetting factor λ	TLS_RO_SOH_block(.m/_openloop.m)	0.997 ($\tau \approx 333$ s memory)	$\lambda = 0.997^{(T_s/1)}$	≈ 0.99970 (same ≈ 333 s memory)
TLS updateEvery (EKF push rate)	TLS_RO_SOH_block(.m/_openloop.m)	round(5/ T_s) = 5 samples	round(5/ T_s)	50 samples (still ~ 5 s)

The λ rescale was missed in an early pass of this redesign: with λ left at 0.997 while T_s moved to 0.1 s, the recursive TLS's real-time memory window silently shrank from ~ 333 s to ~ 33 s (a 10 $\times$ reduction), making the estimator far more sensitive to short, poorly-conditioned stretches of drive-cycle data. This was the root cause of large transient spikes seen in RO_TLS/SOH_ratio early in this project; the fix (rescaling λ the same way as the hysteresis filter, plus a physical-plausibility clamp on candidate RO estimates) is documented in Section 8.

Solver configuration: Fixed-step, discrete (no continuous states), $T_s = 0.1$ s. The From Workspace current-input block's interpolation is set to Zero-Order Hold, consistent with the ZOH-input assumption already built into the exact RC-branch discretization.

4. Algorithms

4.1 2RC discrete Thevenin model

Each RC branch is the exact zero-order-hold discretization of its continuous-time state equation, so it is exact for any T_s under a piecewise-constant (ZOH) current assumption — not an approximation like forward-Euler:

$$V_1(k+1) = a_1 \cdot V_1(k) + R_1 \cdot (1-a_1) \cdot I(k), \quad a_1 = \exp(-T_s / (R_1 \cdot C_1))$$

$$V_2(k+1) = a_2 \cdot V_2(k) + R_2 \cdot (1-a_2) \cdot I(k), \quad a_2 = \exp(-T_s / (R_2 \cdot C_2))$$

$$V_{\text{terminal}}(k) = OCV_{\text{used}}(SOC(k)) - I(k) \cdot R_0 - V_1(k) - V_2(k)$$

$I(k)$ is positive during discharge; $-I \cdot R_0$ and $-V_1-V_2$ correctly reduce terminal voltage below OCV during discharge and increase it during charge.

4.2 Coulomb Counting SOC

$$\text{SOC}(k+1) = \text{SOC}(k) - (\text{Ts} / (\text{Q_nominal} \cdot 3600)) \cdot \text{I}(k) \cdot 100 \quad [\text{SOC in \%}, \text{Q_nominal in Ah}]$$

4.3 OCV + hysteresis noise

$$\text{OCV_used}(t) = \text{OCV_lookup}(\text{SOC}(t)) + \text{HysteresisNoise}(t)$$

HysteresisNoise is Band-Limited White Noise → Discrete Filter (rescaled per Section 3.2) → Saturation (± 0.005 V), per the original spec's FR-3.

4.4 EKF SOC estimator (2RC, 3-state, live R0)

State vector $x = [\text{SOC}; \text{V1}; \text{V2}]$. R0 is no longer a fixed constant — it is an input (R0_in) fed from the TLS identifier, one sample delayed, closing the joint state/parameter estimation loop (this is the same structural pattern as Plett's dual/joint EKF for battery SOC + parameter estimation).

```
function [SOC_hat, V1_hat, V2_hat] = EKF_SOC_Estimator(I_meas, V_meas, R0_in, Ts)
%EKF_SOC_ESTIMATOR Extended Kalman Filter SOC estimator for the Guardian
% Battery ECM Project (2RC Thevenin model, LF50K 3.2V/50Ah LFP cell).
%
% [SOC_hat, V1_hat, V2_hat] = EKF_SOC_Estimator(I_meas, V_meas, R0_in, Ts)
%
% Inputs:
% I_meas - measured cell current (A), discharge = POSITIVE
% V_meas - measured terminal voltage (V)
% R0_in - live R0 estimate (ohm), fed from TLS_R0_SOH_block, ONE
% SAMPLE DELAYED (put a Unit Delay/Memory block on this
% wire in Simulink) -- R0 is no longer a fixed constant
% Ts - sample time (s) -- must match the model's discrete step
%
% Outputs:
% SOC_hat - estimated state of charge (%)
% V1_hat - estimated R1C1-branch polarization voltage (V)
% V2_hat - estimated R2C2-branch polarization voltage (V)
%
% Part 2 Addendum: extended from 1RC/2-state to 2RC/3-state, and R0
% promoted from a fixed constant to a live input from the recursive
% TLS identifier -- this closes the joint state/parameter estimation
% loop with TLS_R0_SOH_block.m.
%#codegen
persistent x P initialized
if isempty(initialized)
    x = [80; 0; 0]; % [SOC(%); V1(V); V2(V)]
    P = diag([4, 0.01, 0.01]); % initial covariance
    initialized = true;
end

% ---- Fixed cell/model parameters (from project datasheet, Appendix A) ----
Q_nominal_Ah = 50;
R1 = 0.00160; % ohm
C1 = 17000; % F
R2 = 0.00035; % ohm
C2 = 5200; % F
a1 = exp(-Ts / (R1*C1));
a2 = exp(-Ts / (R2*C2));

% ---- OCV-SOC breakpoints -- MUST match Guardian_Battery_OCV_SOC_Table.csv ----
soc_bp = [0 5 10 20 30 40 50 60 70 80 90 95 100];
ocv_bp = [2.50 2.95 3.05 3.15 3.20 3.22 3.24 3.26 3.28 3.30 3.33 3.45 3.65];

% ---- Process/measurement noise tuning ----
% Qcov was originally tuned at Ts_ref = 1 s. Discrete process noise
% accumulates roughly linearly with Ts (more, smaller steps per second
% should inject proportionally less noise per step to match the same
% real-time uncertainty growth) -- scaled here, but still a starting
% point to retune/justify like the original two entries were.
Ts_ref = 1;
Qcov = diag([1e-3, 1e-6, 1e-6]) * (Ts / Ts_ref);
Rcov = 4e-4; % sensor noise -- not Ts-dependent, left as-is

% =====
```

```

% PREDICT
% =====
SOC_pred = x(1) - (Ts / (Q_nominal_Ah * 3600)) * I_meas * 100;
V1_pred = a1 * x(2) + R1 * (1 - a1) * I_meas;
V2_pred = a2 * x(3) + R2 * (1 - a2) * I_meas;
x_pred = [SOC_pred; V1_pred; V2_pred];

F = [1, 0, 0;
     0, a1, 0;
     0, 0, a2];
P_pred = F * P * F' + Qcov;

% =====
% UPDATE
% =====
OCV_pred = interp1(soc_bp, ocv_bp, x_pred(1), 'linear', 'extrap');
d = 0.5;
dOCV_dSOC = interp1(soc_bp, ocv_bp, x_pred(1) + d, 'linear', 'extrap') ...
            - interp1(soc_bp, ocv_bp, x_pred(1) - d, 'linear', 'extrap');
dOCV_dSOC = dOCV_dSOC / (2 * d);

H = [dOCV_dSOC, -1, -1];
V_pred = OCV_pred - I_meas * R0_in - x_pred(2) - x_pred(3);
y_resid = V_meas - V_pred;
S = H * P_pred * H' + Rcov;
K = P_pred * H' / S;
x = x_pred + K * y_resid;
P = (eye(3) - K * H) * P_pred;

% Physical bound -- SOC cannot leave [0, 100]%
x(1) = min(max(x(1), 0), 100);

SOC_hat = x(1);
V1_hat = x(2);
V2_hat = x(3);
end

```

EKF_SOC_Estimator.m — full source.

4.5 TLS R0/SOH estimator — Coupled variant

Regresses R0 online using a closed-form, exponentially-weighted recursive Total Least Squares update (equivalent to the batch SVD form used in the original spec, but updated every sample instead of once at the end of a run). This variant sources V1_hat/V2_hat directly from the EKF.

```

function [R0_TLS, SOH_ratio] = TLS_R0_SOH_block(Current_A, V_terminal, SOC_hat, V1_hat, V2_hat, Ts)
%TLS_R0_SOH_BLOCK Recursive (online) TLS identification of R0, closing
% the loop with the EKF -- Part 2 Addendum to Sec. 6.5 / Figure 4.
%
% [R0_TLS, SOH_ratio] = TLS_R0_SOH_block(Current_A, V_terminal, ...
%                                       SOC_hat, V1_hat, V2_hat, Ts)
%
% Inputs:
%   Current_A - measured cell current (A), discharge = POSITIVE
%   V_terminal - measured/simulated terminal voltage (V)
%   SOC_hat - EKF's SOC estimate (%) -- replaces SOC_CC from the
%             original spec so TLS regresses against the same
%             state the EKF tracks, not the open-loop CC value
%   V1_hat - EKF's R1C1 polarization estimate (V) -- replaces
%            this block's own open-loop V1 computation
%   V2_hat - EKF's R2C2 polarization estimate (V)
%   Ts - sample time (s)
%
% Outputs:
%   R0_TLS - live R0 estimate (ohm), rate-limited (held) before
%            being sent to the EKF -- see updateEvery below.
%            Feed this into EKF_SOC_Estimator's R0_in through a
%            Unit Delay/Memory block, not directly.
%   SOH_ratio - R0_datasheet / R0_TLS * 100 (%)
%
% COUPLING NOTE: because V1_hat/V2_hat now come from the same EKF that

```

```

% R0_TLS feeds, an R0 error partially gets absorbed into the EKF's own
% correction step rather than showing up cleanly as residual here.
% updateEvery decouples the two update rates (recursive stats refresh
% every sample; the value actually pushed to the EKF refreshes every
% updateEvery samples) so the EKF has time to resettle between R0
% pushes. Set updateEvery = 1 to disable this and go fully tight-
% coupled if you'd rather observe that behavior directly.
%#codegen
persistent mx my Sxx Sxy Syy R0_running R0_held sampleCount initialized

R0_nominal = 0.00070; % datasheet R0 (ohm) -- seed value + fallback

if isempty(initialized)
    mx = 0; my = 0; Sxx = 0; Sxy = 0; Syy = 0;
    R0_running = R0_nominal;
    R0_held = R0_nominal;
    sampleCount = 0;
    initialized = true;
end

soc_bp = [0 5 10 20 30 40 50 60 70 80 90 95 100];
ocv_bp = [2.50 2.95 3.05 3.15 3.20 3.22 3.24 3.26 3.28 3.30 3.33 3.45 3.65];

% lambda was originally tuned at Ts_ref=1 (tau ~= 333 s real-time memory).
% Rescaled here the same way as hyst_num/hyst_den in the main script --
% left unscaled, Ts=0.1 would silently shrink the memory window to ~33 s,
% making the estimator far more sensitive to short poorly-conditioned
% stretches of data (this was the cause of the R0_TLS transient spikes).
Ts_ref_lambda = 1;
lambda_ref = 0.997;
lambda = lambda_ref ^ (Ts / Ts_ref_lambda);
minSxx = 1e-4; % persistent-excitation guard
minSxyAbs = 1e-8;
updateEvery = max(1, round(5/Ts)); % push a new R0 to the EKF ~every 5 s
R0_min = 0.1 * R0_nominal; % physical-plausibility guard --
R0_max = 5 * R0_nominal; % reject candidates outside a sane range

% ---- regression target/regressor, per Sec. 6.5, now sourced from the EKF ----
OCV_k = interp1(soc_bp, ocv_bp, SOC_hat, 'linear', 'extrap');
y_k = OCV_k - V1_hat - V2_hat - V_terminal;
x_k = Current_A;

% ---- recursive (exponentially-weighted) TLS update -----
mx = lambda*mx + (1-lambda)*x_k;
my = lambda*my + (1-lambda)*y_k;
dx = x_k - mx;
dy = y_k - my;

Sxx = lambda*Sxx + (1-lambda)*dx^2;
Sxy = lambda*Sxy + (1-lambda)*dx*dy;
Syy = lambda*Syy + (1-lambda)*dy^2;

if Sxx > minSxx && abs(Sxy) > minSxyAbs
    D = (Sxx - Syy)^2 + 4*Sxy^2;
    R0_candidate = ((Syy - Sxx) + sqrt(D)) / (2*Sxy); % == -V(1,2)/V(2,2) from the batch SVD form
    if R0_candidate > R0_min && R0_candidate < R0_max
        R0_running = R0_candidate;
    end
    % else: reject as physically implausible, keep the last good R0_running
end

% ---- rate-limit the value actually pushed to the EKF -----
sampleCount = sampleCount + 1;
if sampleCount >= updateEvery
    R0_held = R0_running;
    sampleCount = 0;
end

R0_TLS = R0_held;
SOH_ratio = (R0_nominal / R0_TLS) * 100;
end

```

4.6 TLS R0/SOH estimator — Open-loop variant

Identical recursive TLS core, but V1/V2 are computed internally from the given, known R1/C1/R2/C2 (an independent open-loop RC replica) instead of consuming the EKF's V1_hat/V2_hat. Only R0_TLS feeds back into the EKF — the coupling is one-way. This mirrors the original course spec's Sec. 6.5 design intent, which explicitly warns that skipping the given-R1,C1 polarization removal biases the R0 estimate.

```
function [R0_TLS, SOH_ratio] = TLS_RO_SOH_block_openloop(Current_A, V_terminal, SOC_hat, Ts)
%TLS_RO_SOH_BLOCK_OPENLOOP Recursive (online) TLS identification of R0,
% using OPEN-LOOP V1/V2 computed from the GIVEN R1,C1,R2,C2 -- per
% spec Sec. 6.5's original design intent ("skipping this step biases
% the R0 estimate significantly"), rather than the EKF's own V1_hat/
% V2_hat. Only R0_TLS feeds back into the EKF (one-way coupling);
% this block's own RC-branch replica is independent of the EKF's
% real-time estimation error, so an EKF transient can't leak into
% the R0 regression residual.
%
% [R0_TLS, SOH_ratio] = TLS_RO_SOH_block_openloop(Current_A, ...
%                                                  V_terminal, SOC_hat, Ts)
%
% Inputs:
%   Current_A - measured cell current (A), discharge = POSITIVE
%   V_terminal - measured/simulated terminal voltage (V)
%   SOC_hat - EKF's SOC estimate (%)
%   Ts - sample time (s)
%
% Outputs:
%   R0_TLS - live R0 estimate (ohm), rate-limited (held) before
%           being sent to the EKF -- see updateEvery below.
%           Feed into EKF_SOC_Estimator's R0_in through a Unit
%           Delay/Memory block, same wiring as the coupled
%           version. Note this block no longer takes V1_hat/
%           V2_hat as inputs -- two fewer ports than before.
%   SOH_ratio - R0_datasheet / R0_TLS * 100 (%)
%#codegen
persistent mx my Sxx Sxy Syy R0_running R0_held sampleCount V1_ol V2_ol initialized

R0_nominal = 0.00070; % datasheet R0 (ohm) -- seed value + fallback
R1 = 0.00160; % ohm -- GIVEN, known (not estimated here)
C1 = 17000; % F
R2 = 0.00035; % ohm
C2 = 5200; % F

if isempty(initialized)
    mx = 0; my = 0; Sxx = 0; Sxy = 0; Syy = 0;
    R0_running = R0_nominal;
    R0_held = R0_nominal;
    sampleCount = 0;
    V1_ol = 0; % battery assumed at rest, no polarization, at t=0
    V2_ol = 0;
    initialized = true;
end

soc_bp = [0 5 10 20 30 40 50 60 70 80 90 95 100];
ocv_bp = [2.50 2.95 3.05 3.15 3.20 3.22 3.24 3.26 3.28 3.30 3.33 3.45 3.65];

a1 = exp(-Ts/(R1*C1));
a2 = exp(-Ts/(R2*C2));

% lambda scaled the same way as the coupled version -- see that file's
% comment for why this matters at Ts != 1.
Ts_ref_lambda = 1;
lambda_ref = 0.997;
lambda = lambda_ref ^ (Ts / Ts_ref_lambda);
minSxx = 1e-4; % persistent-excitation guard
minSxyAbs = 1e-8;
updateEvery = max(1, round(5/Ts)); % push a new R0 to the EKF ~every 5 s
R0_min = 0.1 * R0_nominal; % physical-plausibility guard
```

```

R0_max      = 5      * R0_nominal;

% ---- regression target/regressor, per Sec. 6.5, OPEN-LOOP V1/V2 -----
% NOTE: uses V1_ol(k)/V2_ol(k), i.e. the values BEFORE this step's
% advance, matching the index convention in Sec. 6.5.
OCV_k = interp1(soc_bp, ocv_bp, SOC_hat, 'linear', 'extrap');
y_k = OCV_k - V1_ol - V2_ol - V_terminal;
x_k = Current_A;

% ---- THEN advance the open-loop RC replica, given R1,C1,R2,C2 -----
V1_ol = a1*V1_ol + R1*(1-a1)*Current_A;
V2_ol = a2*V2_ol + R2*(1-a2)*Current_A;

% ---- recursive (exponentially-weighted) TLS update -----
mx = lambda*mx + (1-lambda)*x_k;
my = lambda*my + (1-lambda)*y_k;
dx = x_k - mx;
dy = y_k - my;

Sxx = lambda*Sxx + (1-lambda)*dx^2;
Sxy = lambda*Sxy + (1-lambda)*dx*dy;
Syy = lambda*Syy + (1-lambda)*dy^2;

if Sxx > minSxx && abs(Sxy) > minSxyAbs
    D = (Sxx - Syy)^2 + 4*Sxy^2;
    R0_candidate = ((Syy - Sxx) + sqrt(D)) / (2*Sxy); % == -V(1,2)/V(2,2) from the batch SVD form
    if R0_candidate > R0_min && R0_candidate < R0_max
        R0_running = R0_candidate;
    end
    % else: reject as physically implausible, keep the last good R0_running
end

% ---- rate-limit the value actually pushed to the EKF -----
sampleCount = sampleCount + 1;
if sampleCount >= updateEvery
    R0_held = R0_running;
    sampleCount = 0;
end

R0_TLS      = R0_held;
SOH_ratio = (R0_nominal / R0_TLS) * 100;
end

```

TLS_RO_SOH_block_openloop.m — full source (Open-loop variant).

4.7 Variant Subsystem configuration

- Variant control mode: expression.
- Variant activation time: update diagram (compile-time — change the control variable, then re-run; no runtime switching needed for this use case).
- Variant control expressions: TLS_impl_select==1 (Coupled), TLS_impl_select==2 (Open_loop).
- TLS_impl_select is defined in this model's own Model Workspace (not the base MATLAB workspace), so the model resolves its own variant selection correctly regardless of what parent model references it — important since this model is intended to be used as a Model Reference.
- Built-in empty choice: enabled, as a safety net — if TLS_impl_select is ever unset or an unrecognized value, the subsystem resolves to an obviously-empty output rather than an ambiguous pick.

5. Bus Objects

Two distinct bus objects are defined in Guardian_Battery_script.m. They serve different boundaries and are not interchangeable — using the wrong one produces either a data-type mismatch or a bus-hierarchy-mismatch error.

5.1 GuardianCellBus — inner Cell_data bus (8 elements)

Used for Cell_DigitalTwin's internal wiring inside DigitalTwin_and_SOC_SOH. Includes internal-only diagnostic signals not meant to be exposed at the model's public boundary.

#	Element	Data type
1	SOC_CC	double
2	SOC_Flag	boolean
3	SOC_hat	double
4	V1_hat	double
5	V2_hat	double
6	R0_TLS	double
7	SOH_ratio	double
8	V_terminal	double

5.2 BatteryDataBus — public Battery_data boundary (5 elements)

The model's public, model-reference-facing interface — deliberately narrower than GuardianCellBus, exposing only what a caller needs and excluding internal-only diagnostics (SOC_CC, V1_hat, V2_hat).

#	Element	Data type
1	R0_TLS	double
2	SOH_ratio	double
3	SOC_EKF	double
4	SOC_Flag	boolean
5	V_terminal	double

Note the third element is named SOC_EKF here (matching the signal name actually wired into the Bus Creator feeding Battery_data), not SOC_hat — the two buses use different names for the same underlying EKF SOC estimate.

6. Base Workspace Script

Guardian_Battery_script.m loads the drive-cycle and OCV data, defines every model parameter (no magic numbers in the model itself), computes the Ts-rescaled hysteresis filter coefficients, and defines both bus objects. Run this before opening or simulating the model.

```
close all
clc
clearvars
%% Current and time

driveCycleTbl = readtable('Guardian_Battery_DriveCycle_Current (1)');
ocvLookupTbl = readtable('Guardian_Battery_OCV_SOC_Table (1)');

t_vec_s = driveCycleTbl.Time_s;
```

```

I_drive_A = driveCycleTbl.Current_A;

SOC_breakpoints = ocvLookupTbl.SOC_pct;
OCV_data = ocvLookupTbl.OCV_V;

I_workspace = [t_vec_s , I_drive_A];

T_end_s = max(t_vec_s);

%% Constants

Ts = 0.1;
Q_nominal = 50; % Ah
R0_ref = 0.0007; % ohm
R1_ref = 0.0016; % ohm
C1_ref = 17000; % F
R2_ref = 0.00035; % ohm
C2_ref = 5200; % F
tau1_s = R1_ref * C1_ref;
tau2_s = R2_ref * C2_ref;
a1_coef = exp(-Ts/(tau1_s));
a2_coef = exp(-Ts/(tau2_s));
SOC_init_pct = 80;

%% Hysteresis Noise Filter (FR-3, rescaled for Ts)
% FR-3 specifies this filter at Ts_ref = 1 s: Numerator = [0.02],
% Denominator = [1 -0.98] (tau ~= 49.5 s, unity DC gain). Rescaled here
% to preserve that same physical bandwidth at the model's actual Ts --
% left as Ts=1 while everything else moves to 0.1 would make the
% hysteresis noise ~10x jitterier than the reference realization.
% Noise Power (1e-4) and Saturation limits (+-0.005) are Ts-independent
% and stay as FR-3 specifies them.

Ts_hyst_ref = 1;
a_hyst_baseline = 0.98;
a_hyst_coef = a_hyst_baseline ^ (Ts / Ts_hyst_ref);
hyst_filt_num = 1 - a_hyst_coef;
hyst_filt_den = [1, -a_hyst_coef];

% Apply in Simulink:
%   Band-Limited White Noise block -> Sample time = Ts
%   Discrete Filter block           -> Numerator = hyst_filt_num,
%                                     Denominator = hyst_filt_den

%% Bus

busElems(1) = Simulink.BusElement;
busElems(1).Name = 'SOC_CC'; busElems(1).DataType = 'double';

busElems(2) = Simulink.BusElement;
busElems(2).Name = 'SOC_Flag'; busElems(2).DataType = 'boolean';

busElems(3) = Simulink.BusElement;
busElems(3).Name = 'SOC_hat'; busElems(3).DataType = 'double';

busElems(4) = Simulink.BusElement;
busElems(4).Name = 'V1_hat'; busElems(4).DataType = 'double';

busElems(5) = Simulink.BusElement;
busElems(5).Name = 'V2_hat'; busElems(5).DataType = 'double';

busElems(6) = Simulink.BusElement;
busElems(6).Name = 'R0_TLS'; busElems(6).DataType = 'double';

busElems(7) = Simulink.BusElement;
busElems(7).Name = 'SOH_ratio'; busElems(7).DataType = 'double';

busElems(8) = Simulink.BusElement;
busElems(8).Name = 'V_terminal'; busElems(8).DataType = 'double';

```

```

GuardianCellBus = Simulink.Bus;
GuardianCellBus.Elements = busElems;
% GuardianCellBus is the INNER Cell_data bus (Cell_DigitalTwin's own
% wiring) -- 8 elements including internal-only diagnostics (SOC_CC,
% V1_hat, V2_hat). It is NOT the right bus object for the outer
% Battery_data port -- see BatteryDataBus below for that boundary.

%% Bus (public -- Battery_data output)
% Narrower, 5-element interface matching what's actually wired into the
% Battery_data output -- the boundary this referenced model exposes to
% whatever calls it, so it deliberately excludes internal-only signals
% (SOC_CC, V1_hat, V2_hat) that GuardianCellBus carries for wiring
% inside DigitalTwin_and_SOC_SOH.

busElemsPublic(1) = Simulink.BusElement;
busElemsPublic(1).Name = 'R0_TLS'; busElemsPublic(1).DataType = 'double';

busElemsPublic(2) = Simulink.BusElement;
busElemsPublic(2).Name = 'SOH_ratio'; busElemsPublic(2).DataType = 'double';

busElemsPublic(3) = Simulink.BusElement;
busElemsPublic(3).Name = 'SOC_EKF'; busElemsPublic(3).DataType = 'double';

busElemsPublic(4) = Simulink.BusElement;
busElemsPublic(4).Name = 'SOC_Flag'; busElemsPublic(4).DataType = 'boolean';

busElemsPublic(5) = Simulink.BusElement;
busElemsPublic(5).Name = 'V_terminal'; busElemsPublic(5).DataType = 'double';

BatteryDataBus = Simulink.Bus;
BatteryDataBus.Elements = busElemsPublic;

%% Plots

% subplot( 4 , 4 , [1 5 9 13])
% plot(t_vec_s , out.OCV_used.signals.values , Color = "b")
% xlabel('Time (s)')
% ylabel('OCV USED (V)')
% grid on
%
% subplot( 4 , 4 , [2 6 10 14])
% plot(t_vec_s , out.V1.signals.values , Color = "g")
% xlabel('Time (s)')
% ylabel('V1 (V)')
% grid on
%
% subplot( 4 , 4 , [3 7 11 15])
% plot(t_vec_s , out.SOC_CC.signals.values , Color = "r")
% xlabel('Time (s)')
% ylabel('SOC CC (%)')
% grid on
%
% subplot( 4 , 4 , [4 8 12 16])
% plot(t_vec_s , out.Terminal_Voltage.signals.values , Color = "c")
% xlabel('Time (s)')
% ylabel('Voltage TERMINAL (V)')
% grid on

```

Guardian_Battery_script.m — full source.

7. Validation Results

Results below are from a full 3600 s drive-cycle run at $T_s = 0.1$ s, $SOC(0) = 80\%$, using the Open_loop TLS variant (TLS_impl_select = 2) unless noted.

7.1 SOC: Coulomb Counting vs. EKF

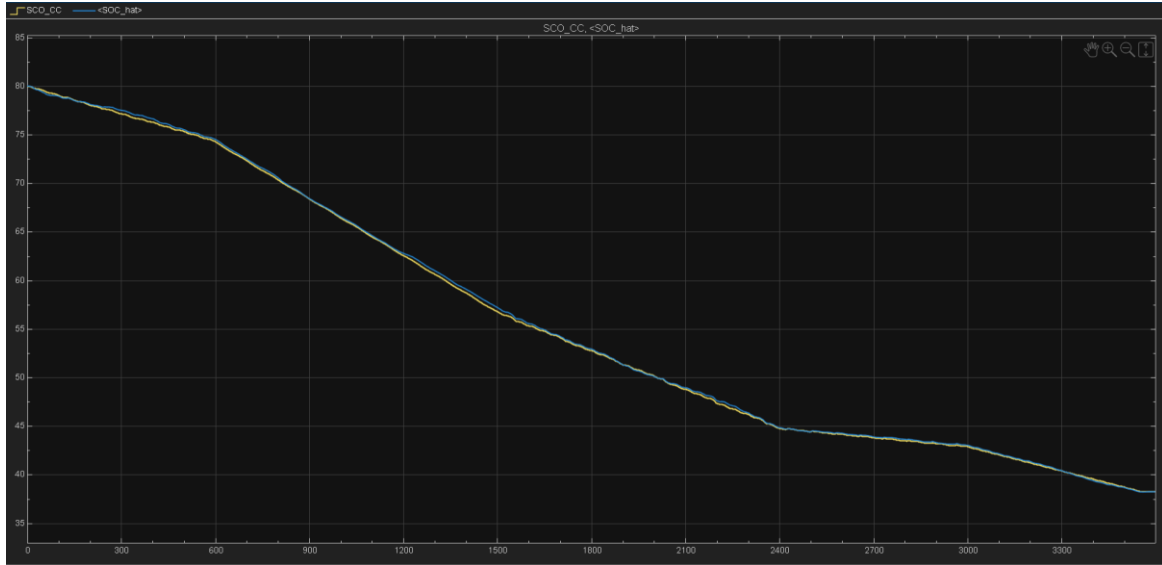


Figure 9. SOC_CC (Coulomb Counting) vs. SOC_hat (EKF) over the full drive cycle. The two track within a fraction of a percent throughout, confirming the 2RC extension and the R0 feedback loop are not corrupting the core SOC estimate.

7.2 R0 identification

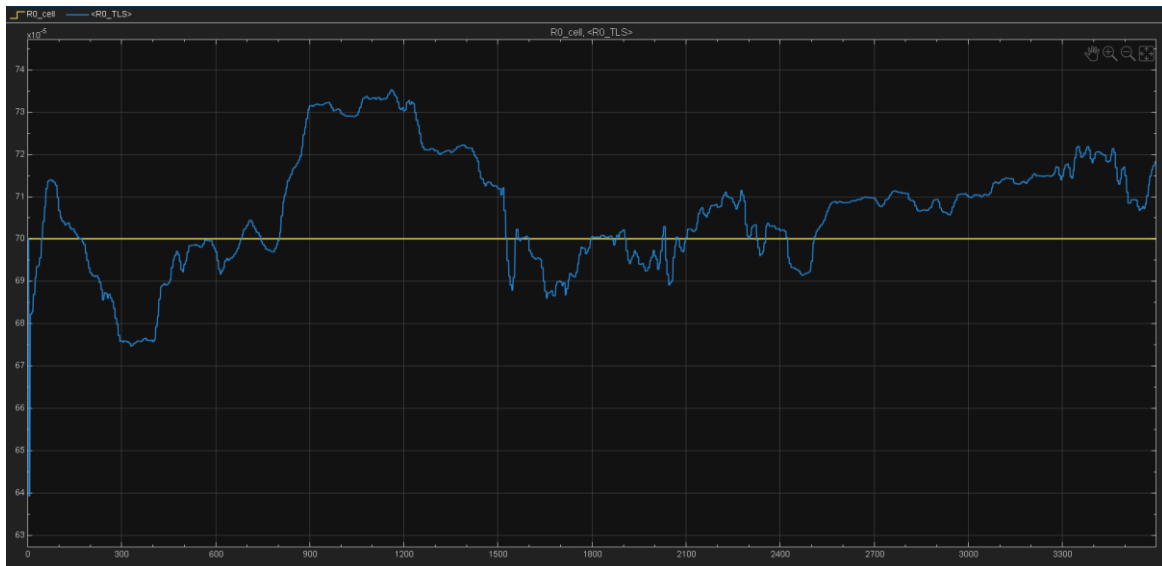


Figure 10. R0_TLS (blue) vs. the constant reference $R0_{ref} = 0.0007 \Omega$ (yellow line). R0_TLS converges to and holds close to the true value for the full run; the visible variation is the residual noise floor after the λ rescale and physical-plausibility clamp fixes (Section 8), not divergence.

7.3 SOH ratio

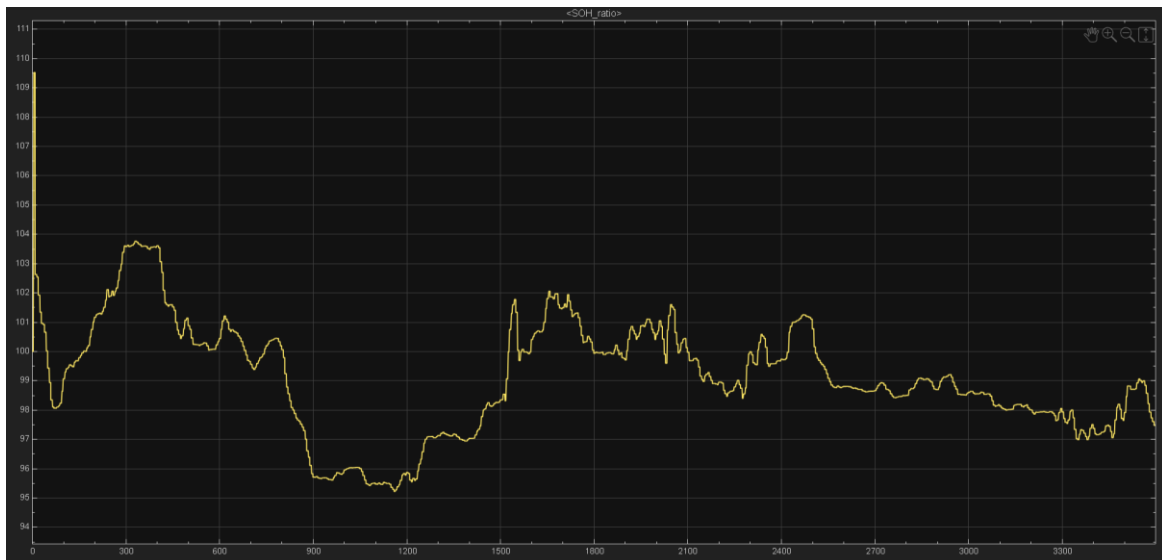


Figure 11. $SOH_ratio = R0_ref / R0_TLS \times 100\%$, Open-loop variant. Settles into roughly a 95–104% band after the initial startup transient — within the course rubric's 90–105% healthy-cell acceptance band.

7.4 Coupled vs. Open-loop — observed result and analysis

Contrary to the initial prediction (Section 4.6), the Open-loop variant's SOH_ratio in this run is not tighter than the Coupled variant's — it is comparable to or wider than it. The Coupled variant (measured in an earlier run at $T_s=0.1$ s, after the λ and clamp fixes but before the noise-block T_s fix) settled to roughly a 97.8–103% band; the Open-loop variant here (Fig. 11) swings roughly 95–104%, with a larger startup transient.

Working hypothesis, not confirmed: the Coupled variant's $V1_hat/V2_hat$ are themselves EKF-filtered states, so the EKF's own Kalman correction step may be inadvertently smoothing some of the hysteresis noise out of the signal TLS regresses against — making the Coupled variant's $R0_TLS$ look less noisy in this specific test, without that necessarily meaning it is more accurate (recall the Coupled variant carries a structural bias the Open-loop variant was specifically built to remove, per Sec. 6.5's warning — see Section 8.3). The Open-loop variant's own RC replica is an exact, noiseless copy of the plant's true $V1/V2$ in this simulation (same $R1/C1/R2/C2$, same current), so it fully exposes the hysteresis noise floor in the regression residual instead of having it partially absorbed by a Kalman correction step.

This reads as a bias-vs-variance tradeoff rather than one variant being strictly better: Coupled trades a small structural bias for a visually tighter estimate; Open-loop removes that structural bias but shows the underlying measurement/hysteresis noise more directly. Both variants stayed within the course rubric's 90–105% band in every run to date. A natural next step, not yet tried: increase λ and/or updateEvery specifically for the Open-loop variant, to average over more of that now fully-exposed noise without reintroducing the Coupled variant's structural bias.

8. Development History and Key Fixes

A running log of the non-obvious issues found and fixed during this redesign, kept here so they don't need to be rediscovered.

8.1 Bus object mismatch after variable renaming

Renaming Guardian_Battery_script.m's variables (including CellDataBus → GuardianCellBus) broke any Simulink block still referencing the old name by string, producing “expects double, driven by

mixed type” errors at the Battery_data output. Root cause: Battery_data's Data type field was pointed at a bus object name that no longer existed. Fix: point it at the correct, currently-defined bus object, and click Apply/OK to actually commit the change (a dialog left open without committing will keep showing the old, broken behavior on the next run).

8.2 Bus hierarchy mismatch

GuardianCellBus (8 elements, for the inner Cell_data wiring) was briefly applied to Battery_data, whose Bus Creator only carries 5 signals in a different order (R0_TLS, SOH_ratio, SOC_EKF, SOC_Flag, V_terminal) — hence “hierarchy of the input bus must match.” Fix: a separate, narrower bus object, BatteryDataBus, matching exactly what's wired into that specific boundary (Section 5.2).

8.3 Recursive TLS forgetting factor not rescaled for Ts

Covered in detail in Section 3.2. Qcov, the hysteresis filter, and updateEvery were all correctly rescaled when Ts moved from 1 s to 0.1 s; λ was initially missed. The fix rescales λ the same way, plus adds a physical-plausibility clamp (reject any candidate R0 outside $[0.1, 5] \times R0_nominal$) as a second, independent safety net against transient blow-ups.

8.4 R0_cell is a comparison signal only, not part of the estimation loop

R0_cell (Fig. 7, external input) is a separately-computed reference/diagnostic signal used only for visually comparing against R0_TLS on a scope — it is not wired into the EKF or TLS_RO_SOH. Early in this project, a naive instantaneous V/I-style comparison signal was mistaken for ground truth; the actual ground truth for validating R0_TLS is the constant $R0_ref = 0.0007 \, \Omega$ defined in the workspace script.

8.5 Fast Restart and dialog changes not taking effect

Simulink dialog fields only take effect after Apply/OK; if Fast Restart is enabled, a fresh compile may not occur between runs even after a setting changes. When a fix doesn't appear to take effect, disable Fast Restart, clear diagnostics, force Update Diagram (Ctrl+D), and re-run before concluding the fix didn't work.